

Quiz 1: CS228 2026

Time: 75 min, Marks: 35

Aug 28 2026

1. (10 marks) For each of the following questions answer true or false. You must also provide a short but complete justification.

- (a) The formulas $(q \vee \neg q) \rightarrow \neg p$ and $p \rightarrow (q \wedge \neg q)$ are equivalent.

Solution

True.

p	q	$\neg p$	$\neg q$	$q \vee \neg q$	$q \wedge \neg q$	$(q \vee \neg q) \rightarrow \neg p$	$p \rightarrow (q \wedge \neg q)$
T	T	F	F	T	F	F	F
T	F	F	T	T	F	F	F
F	T	T	F	T	F	T	T
F	F	T	T	T	F	T	T

Note here that the last two columns match. Hence the two formulae are equivalent.

Grading Scheme:

0.5 marks for writing that it is True

1.5 marks for correct justification. Marks have been given even if truth table is not shown, even though it was expected. Marks have also been given for arguing that the statements are contrapositive of each other

Common Mistake: Copying the formula from the question wrong.

- (b) $((\neg p \vee q) \wedge (\neg r \wedge \neg p))$ is satisfiable but not valid.

Solution

True.

A satisfying assignment: $p \mapsto \perp, q \mapsto \perp, r \mapsto \perp$ ($q \mapsto \top$ also works)

A falsifying assignment: $p \mapsto \top, q \mapsto \perp, r \mapsto \top$ ($r \mapsto \perp$ will also work).

Grading Scheme:

0.5 marks for writing that there exists a satisfying assignment

0.5 marks for giving a correct satisfying assignment

0.5 marks for writing that there exists a falsifying assignment

0.5 marks for giving a correct falsifying assignment

- (c) If F and G are both valid propositional formulae, then $\neg(F \wedge G)$ must be satisfiable.

Solution

False. Consider $F = \top, G = \top$. Both are valid formulae. But $\neg(F \wedge G) = \neg(\top \wedge \top) = \perp$ is not satisfiable.

Grading Scheme:

0.5 marks for writing that it is False

1.5 marks for correct justification. Marks have been given to those who have shown this using the lemma proved in class as well.

Common mistake: Not considering the fact that F and G are valid formulae, and giving a satisfiable assignment for $\neg(F \wedge G)$ considering an assignment where one of F or G evaluate to 0.

- (d) If F, G , and H are propositional formulae such that $F, G \models H$ holds, then $F \vdash (G \rightarrow H)$ is a valid sequent.

Solution

True. Using completeness of propositional logic, from $F, G \models H$ we get that $F, G \vdash H$.

1.	F	premise
2.	G	assumption
3.	H	using $F, G \vdash H$
4.	$G \rightarrow H$	$\rightarrow i$ 2-3

Grading Scheme:

0.5 marks for writing that it is True

1.5 marks for correct justification. In this, 0.5 marks are allotted for mentioning the completeness of propositional logic. 1 mark is given for a correct natural deduction proof as shown, or for arguing semantically that if $F, G \models H$, then $F \models (G \rightarrow H)$. Although we expected a proper ND proof, we have given marks to those who stated how to convert the proof of $F, G \vdash H$ to a proof of $F \vdash (G \rightarrow H)$ without giving the proof itself. Minor mistakes like not writing the proof rule next to each line, or missing a case while arguing semantically, have been excused. However, those who used ND to argue about semantic entailment, or a semantic argument to talk about validity of sequents, have lost marks.

Common mistakes: Forgetting the importance of completeness of propositional logic. Confusing soundness and completeness (note that correctness is the same as soundness, which is not the same as completeness!).

- (e) The formula $F \wedge (G \vee H)$ has at least three different equivalent formulae which are all in DNF, and do not have $\perp$ (or any subformula equivalent to $\perp$). Note that, $F \wedge G$ and $G \wedge F$, as well as $F \vee G$ and $G \vee F$, are identical, hence not counted as different.

Solution

True. Here are four formulae:

- $(F \wedge G) \vee (F \wedge H)$,
- $(F \wedge G \wedge H) \vee (F \wedge G \wedge \neg H) \vee (F \wedge H)$,
- $(F \wedge G) \vee (F \wedge H \wedge G) \vee (F \wedge H \wedge \neg G)$.
- $(F \wedge G) \vee (F \wedge G \wedge H) \vee (F \wedge H)$

Grading Scheme:

0.5 marks for writing that it is True

0.5 marks for each correct formula

Marks have not been given for formulae which used the trivial constraints $\perp, \top$, as well as those that use idempotence (e.g. $(F \vee F \vee G) \wedge (F \vee F \vee H)$ or $(F \vee G) \wedge (F \vee H) \wedge (F \vee H)$)

2. (5 marks) (**Server Outage**) We are given the following statements:

- If the main server crashes, the backup server starts.

- If the backup server starts, the error is logged and the admin is notified.
- It is strictly false that both the main server crashed and the error was logged.
- If the main server did not crash, the daily report is generated.

Encode the above statements into propositional logic formulae, using appropriate propositional variables. Using the above statements, can you deduce the following propositions using Natural Deduction, or give a counterexample: (a) The backup server started, (b) The main server crashed, (c) The daily report was generated.

Solution

Let M = Main server crashes, B = Backup server starts, L = Error is logged, A = Admin is notified, D = Daily report is generated.

The premises are: $M \rightarrow B$; $B \rightarrow (L \wedge A)$; $\neg(M \wedge L)$; $\neg M \rightarrow D$

We first show that B is not forced by the premises. Take the assignment:

$$M = 0, \quad B = 0, \quad L = 0, \quad A = 0, \quad D = 1$$

All the premises are true under this assignment, but B is false. Hence, the backup server did not necessarily start.

Now we prove that the main server did not crash ($\neg M$) and the daily report is generated (D).

1.	$M \rightarrow B$	premise
2.	$B \rightarrow (L \wedge A)$	premise
3.	$\neg(M \wedge L)$	premise
4.	$\neg M \rightarrow D$	premise
5.	M	assumption
6.	B	MP 1, 5
7.	$L \wedge A$	MP 2, 6
8.	L	$\wedge e_1$ 7
9.	$M \wedge L$	$\wedge i$ 5, 8
10.	$\perp$	$\neg e$ 9, 3
11.	$\neg M$	$\neg i$ 5–10
12.	D	MP 4, 11

Therefore,

We cannot prove the backup server started, but the main server did not crash and the report is generated.

Grading Scheme:

- 1 mark** total for all 4 correct encodings.
Deduct 0.5 marks for using semantic entailment in 3rd encoding.
Deduct 0.5 marks for at most 2 incorrect encodings.
No marks for 3 or more incorrect encodings.
- Any mistake in any encoding - If counter-assignments are correct for (a) & (b) and ND Proof is correct: **at most 3 marks** depending on mistakes in encoding.
- 1 mark** for correct assignment showing (a) is false while all the premises are true, otherwise 0 marks.
- 1 mark** to either derive or give a complete assignment showing (b) is false, otherwise 0 for this part.
- 2 marks** to derive (c) is true
- Deduct 0.5 marks for applying De-Morgan's in 3rd premise during ND proof

- (vii) No ND proof given to show (c) is true gives 0 instead of 2 marks. 0.5 marks can be given if they show complete set of correct arguments.
- (viii) If they show either (b) is true or (c) is false - deduct 1 or 2 depending on the particular result.
- (ix) Deduct no more than 1 mark for any mistake in the ND proof.

3. (10 marks) Are the following sequents valid? If yes, prove them by writing ND proofs (using any of the 12 rules, 4 derived rules and the copy rule) of at most 25 lines, and if not, give an assignment which violates it. Partial marks will be given if your proof has more than 25 but fewer than 35 lines – of course, you must number the lines of your proof.

- (a) $\neg(p \rightarrow q) \vdash p \wedge \neg q$
- (b) $(p \rightarrow (q \rightarrow r)) \vdash ((p \rightarrow (r \rightarrow q)))$
- (c) $\vdash (p \rightarrow (q \vee r)) \rightarrow ((p \rightarrow q) \vee (p \rightarrow r))$

Solution

1.	$\neg(p \rightarrow q)$	premise
2.	$\neg p$	assumption
3.	p	assumption
4.	$\perp$	$\neg e$ 3, 2
5.	q	$\perp e$ 4
6.	$p \rightarrow q$	$\rightarrow i$ 3–5
7.	$\perp$	$\neg e$ 6, 1
(a) 8.	p	$\neg \neg e$ 2–7
9.	q	assumption
10.	p	assumption
11.	q	copy 9
12.	$p \rightarrow q$	$\rightarrow i$ 10–11
13.	$\perp$	$\neg e$ 12, 1
14.	$\neg q$	$\neg i$ 9–13
15.	$p \wedge \neg q$	$\wedge i$ 8, 14

Grading Scheme:

- **2.5 marks** for correct proof,
- **deduct 0.5** for 1-2 wrong rule names,
- **deduct 1** for 4-5 wrong rule names,
- **deduct 0.5** if the proof exceeds 25 lines,
- Wrong / invalid formula deduced inside a proof -
 - **0** if the overall proof is invalid without this step,
 - **1** mark given if overall proof is complete without this intermediate step,
- **0** marks for incomplete proof.

- (b) This sequent is invalid. Consider the assignment $p \mapsto 1, q \mapsto 0, r \mapsto 1$. This makes the left-hand side True, while the right-hand side evaluates to False. Thus the entailment does not hold, and hence by (the contrapositive of) soundness we know that the sequent is not valid.

Common mistake: The assignment $p \mapsto 1, q \mapsto 1, r \mapsto 0$ was given as a counter-example. This is **wrong**. Since the left hand side evaluates to False, the semantic entailment still holds for this assignment, and soundness is not violated.

1.	$p \rightarrow (q \vee r)$	assumption
2.	$p \vee \neg p$	LEM
3.	p	assumption
4.	$q \vee r$	MP 1, 3
5.	q	assumption
6.	p	assumption
7.	q	copy 5
8.	$p \rightarrow q$	$\rightarrow i$ 6-7
9.	$(p \rightarrow q) \vee (p \rightarrow r)$	$\vee i_1$ 8
10.	r	assumption
11.	p	assumption
(c) 12.	r	copy 10
13.	$p \rightarrow r$	$\rightarrow i$ 11-12
14.	$(p \rightarrow q) \vee (p \rightarrow r)$	$\vee i_2$ 13
15.	$(p \rightarrow q) \vee (p \rightarrow r)$	$\vee e$ 4, 5-9, 10-14
16.	$\neg p$	assumption
17.	p	assumption
18.	$\perp$	$\neg e$ 17, 16
19.	q	$\perp e$ 18
20.	$p \rightarrow q$	$\rightarrow i$ 17-19
21.	$(p \rightarrow q) \vee (p \rightarrow r)$	$\vee i_1$ 20
22.	$(p \rightarrow q) \vee (p \rightarrow r)$	$\vee e$ 2, 3-15, 16-21
23.	$(p \rightarrow (q \vee r)) \rightarrow ((p \rightarrow q) \vee (p \rightarrow r))$	$\rightarrow i$ 1-22

Grading Scheme:

- **5 marks** for correct proof,
- **deduct 1** for 1-2 wrong rule names,
- **deduct 2** for 3-5 wrong rule names,
- **deduct 1** if the proof exceeds 25 lines but within 30 lines,
- **deduct 2** if the proof exceeds 30 lines,
- Wrong / invalid formula deduced inside a proof -
 - **0** if the overall proof is invalid without this step,
 - **1** mark given if overall proof is complete without this intermediate step,
- **0** marks for incomplete proof.

Common Mistakes:

- Many students forgot to write the last step $\rightarrow i$. Only 1 mark has been deducted if the rest of the proof is correct.
- If lines 6-7 and 11-12 were skipped (or their equivalent), 2 marks were deducted.

4. (10 marks) Consider $F := \neg(p \wedge q) \rightarrow \neg r$.

(a) Convert F into NNF, CNF and DNF.

Solution

NNF:

$$\begin{aligned} F &\equiv \neg(\neg(p \wedge q)) \vee \neg r \\ &\equiv (p \wedge q) \vee \neg r \end{aligned}$$

CNF:

$$F_{NNF} \equiv (p \wedge q) \vee \neg r$$

$$F_{CNF} \equiv (p \vee \neg r) \wedge (q \vee \neg r)$$

DNF:

$$F_{CNF} \equiv (p \vee \neg r) \wedge (q \vee \neg r)$$

$$F_{DNF} \equiv (p \wedge q) \vee \neg r$$

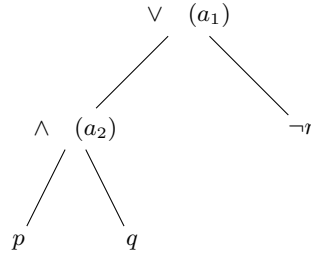
Grading Scheme:

- 1 mark for each correct NNF, CNF and DNF. 0 marks for each wrong one.

(b) Let F_2 denote the NNF formula obtained above (by converting F_1 to NNF). Convert F_2 into an equisatisfiable CNF formula F_3 using Tseitin's encoding via the algorithm done in class. (You must have an auxiliary variable at the root called a_1 . You need not introduce new variables for negations, but no other optimizations are allowed.)

Solution

Convert G to an equisatisfiable CNF formula F' using Tseitin's encoding
Let the NNF formula be $G = (p \wedge q) \vee \neg r$.



We introduce fresh auxiliary variables for each logical connective (subformula) in G :

- Let a_2 represent the inner subformula: $p \wedge q$
- Let a_1 represent the root subformula: $a_2 \vee \neg r$

Tseitin's encoding constructs a formula that asserts the root variable is true, and logically connects each auxiliary variable to its subformula using a biconditional ($\leftrightarrow$).

The formula before converting to CNF is:

$$F' = a_1 \wedge (a_2 \leftrightarrow (p \wedge q)) \wedge (a_1 \leftrightarrow (a_2 \vee \neg r))$$

Next, we convert each biconditional into CNF clauses:

1. Clauses for $a_2 \leftrightarrow (p \wedge q)$:

$$\begin{aligned} a_2 \leftrightarrow (p \wedge q) &\equiv (a_2 \rightarrow (p \wedge q)) \wedge ((p \wedge q) \rightarrow a_2) \\ &\equiv (\neg a_2 \vee (p \wedge q)) \wedge (\neg(p \wedge q) \vee a_2) \\ &\equiv (\neg a_2 \vee p) \wedge (\neg a_2 \vee q) \wedge (\neg p \vee \neg q \vee a_2) \end{aligned}$$

2. Clauses for $a_1 \leftrightarrow (a_2 \vee \neg r)$:

$$\begin{aligned} a_1 \leftrightarrow (a_2 \vee \neg r) &\equiv (a_1 \rightarrow (a_2 \vee \neg r)) \wedge ((a_2 \vee \neg r) \rightarrow a_1) \\ &\equiv (\neg a_1 \vee a_2 \vee \neg r) \wedge (\neg(a_2 \vee \neg r) \vee a_1) \\ &\equiv (\neg a_1 \vee a_2 \vee \neg r) \wedge ((\neg a_2 \wedge r) \vee a_1) \\ &\equiv (\neg a_1 \vee a_2 \vee \neg r) \wedge (\neg a_2 \vee a_1) \wedge (r \vee a_1) \end{aligned}$$

Final Tseitin Encoded CNF (F'):

By conjoining the root unit clause and all the derived clauses, we get:

$$F' = a_1 \wedge (\neg a_2 \vee p) \wedge (\neg a_2 \vee q) \wedge (\neg p \vee \neg q \vee a_2) \wedge (\neg a_1 \vee a_2 \vee \neg r) \wedge (\neg a_2 \vee a_1) \wedge (r \vee a_1)$$

Note: F' has a total of 7 clauses.

Grading Scheme:

- 0.5 marks for parse tree, biimplications and a_1 at the root
- 0.5 marks for CNF form of $a_1 \leftrightarrow \phi_2$
- 0.5 marks for CNF form of $a_2 \leftrightarrow \phi_2$
- 0.5 marks for final equisatisfiable CNF

(c) Now consider an optimization of Tseitin's encoding as follows. For any auxiliary variable a , for which there is a constraint $a \leftrightarrow \psi$, if a occurs only positively (un-negated) wherever it is used, then we keep only $a \rightarrow \psi$ (and drop $a \leftarrow \psi$). Similarly, if it occurs only negatively, we keep only $a \leftarrow \psi$. If a occurs both positively and negatively, then we keep both directions (i.e., retain usual Tseitin encoding).

- i. Apply this optimized version of Tseitin encoding to the NNF formula F_2 above and show the CNF formula obtained. Does it have fewer clauses than F_3 ?
- ii. In general, if G is an arbitrary propositional formula in NNF, does this optimization preserve equi-satisfiability? Why or why not? Give a formal proof.

Solution

i. Applying the optimized encoding to G

Recall our parse tree for $G = (p \wedge q) \vee \neg r$ using 2 auxiliary variables:

- a_2 represents $p \wedge q$
- a_1 represents $a_2 \vee \neg r$ (the root)

We determine the polarity of each auxiliary variable by looking at how it appears in the formula containing it:

- **Root variable (a_1):** It appears as a positive unit clause (a_1). Thus, it has only **positive** polarity. We keep only $a_1 \rightarrow (a_2 \vee \neg r)$.
- **Inner variable (a_2):** In its parent's expression $a_2 \vee \neg r$, a_2 appears without any negation. Thus, it also has only **positive** polarity. We keep only $a_2 \rightarrow (p \wedge q)$.

Converting these retained implications into CNF:

$$a_1 \rightarrow (a_2 \vee \neg r) \equiv \neg a_1 \vee a_2 \vee \neg r$$

$$a_2 \rightarrow (p \wedge q) \equiv (\neg a_2 \vee p) \wedge (\neg a_2 \vee q)$$

Optimized CNF formula (F'_{opt}):

$$F'_{opt} = a_1 \wedge (\neg a_1 \vee a_2 \vee \neg r) \wedge (\neg a_2 \vee p) \wedge (\neg a_2 \vee q)$$

Does it have fewer clauses than F' ?

Yes. The optimized formula F'_{opt} has **4 clauses**, whereas the standard Tseitin encoding F' had **7 clauses**.

Grading Scheme:

- 1 marks if completely correct.
- 0 marks if optimization applied wrong or final formula not an equisatisfiable CNF

ii. Equi-satisfiability for NNF formulas

Yes, this optimization preserves equi-satisfiability.

Proof:

Let F be a formula in NNF. A key property of NNF is that negations are only applied to atomic variables. There are no negations applied to compound subformulas. Therefore, every auxiliary variable a_ψ introduced for an internal subformula (an AND or an OR gate) will appear **strictly positively** in the expression of its parent.

Because all auxiliary variables have only positive polarity, the optimized encoding F_{opt} strictly consists of the root unit clause a_F and implications of the form $a_\psi \rightarrow E(\psi)$.

We must show that F is satisfiable if and only if F_{opt} is satisfiable.

1. Forward Direction (F is satisfiable $\implies F_{opt}$ is satisfiable):

Assume F is satisfiable. There exists a truth assignment V to the original variables that makes F evaluate to True. Extend this assignment to V' by assigning every auxiliary variable a_ψ the exact truth value of its corresponding subformula ψ . By definition, this assignment perfectly satisfies the standard Tseitin encoding (the biconditionals $a_\psi \leftrightarrow E(\psi)$). Since F_{opt} is simply a subset of the clauses in the standard encoding, V' automatically satisfies F_{opt} .

2. Reverse Direction (F_{opt} is satisfiable $\implies F$ is satisfiable):

Assume F_{opt} is satisfiable. Let V' be a satisfying assignment for F_{opt} . Let V be the restriction of V' strictly to the original atomic variables. We will prove by structural induction on the subformulas ψ that:

$$\text{If } V'(a_\psi) = \text{True, then } V(\psi) = \text{True.}$$

- **Base Case (Literals):** If ψ is a literal l (a variable or its negation), it has no auxiliary variable. The condition trivially holds because $V(l) = V'(l)$.
- **Inductive Step (AND):** Suppose $\psi = \psi_1 \wedge \psi_2$. Because F is in NNF, the encoding contains the positive-polarity rule $a_\psi \rightarrow (a_{\psi_1} \wedge a_{\psi_2})$.
If $V'(a_\psi) = \text{True}$, the implication forces both $V'(a_{\psi_1}) = \text{True}$ and $V'(a_{\psi_2}) = \text{True}$. By the inductive hypothesis, $V(\psi_1) = \text{True}$ and $V(\psi_2) = \text{True}$. Therefore, $V(\psi_1 \wedge \psi_2) = \text{True}$.
- **Inductive Step (OR):** Suppose $\psi = \psi_1 \vee \psi_2$. The encoding contains the rule $a_\psi \rightarrow (a_{\psi_1} \vee a_{\psi_2})$.
If $V'(a_\psi) = \text{True}$, the implication forces either $V'(a_{\psi_1}) = \text{True}$ or $V'(a_{\psi_2}) = \text{True}$. By the inductive hypothesis, either $V(\psi_1) = \text{True}$ or $V(\psi_2) = \text{True}$. Therefore, $V(\psi_1 \vee \psi_2) = \text{True}$.

Since V' satisfies F_{opt} , the unit clause for the root variable a_F is satisfied, meaning $V'(a_F) = \text{True}$. By our proven induction, $V'(a_F) = \text{True} \implies V(F) = \text{True}$. Thus, the original assignment V satisfies F . ■

ii. Equi-satisfiability for NNF Formulas (Using Monotonicity)

Yes, this optimization preserves equi-satisfiability for formulas in NNF.

To rigorously prove this, we will first define truth values numerically such that False = 0 and True = 1. The logical implication $A \rightarrow B$ is then equivalent to the inequality $A \leq B$.

Because a formula in NNF restricts all NOT gates ($\neg$) to the leaves (applied only to atomic variables), the internal structure of the formula is built entirely from AND ($\wedge$) and OR ($\vee$) gates.

We first prove a lemma about the monotonic behavior of these operators.

Lemma: Monotonicity of NNF Operators

Statement: Let ψ be a Boolean formula constructed solely of $\wedge$ and $\vee$ operators applied to some inputs. If we have two sets of truth values for these inputs, U and W , such that every input $u_i \in U$ is less than or equal to its corresponding input $w_i \in W$ (i.e., $u_i \leq w_i$), then the evaluation of the formula under U is less than or equal to the evaluation under W : $\psi(U) \leq \psi(W)$.

Proof by structural induction on ψ :

- **Base Case (Input):** ψ is simply a single input x_i . By our premise, $u_i \leq w_i$, so the condition trivially holds.
- **Inductive Step ($\wedge$):** Suppose $\psi = \phi_1 \wedge \phi_2$. By the inductive hypothesis, assume $\phi_1(U) \leq \phi_1(W)$ and $\phi_2(U) \leq \phi_2(W)$.
The logical AND operator computes the minimum of its inputs.

$$\psi(U) = \min(\phi_1(U), \phi_2(U))$$

Since $\phi_1(U) \leq \phi_1(W)$ and $\phi_2(U) \leq \phi_2(W)$, it mathematically follows that:

$$\min(\phi_1(U), \phi_2(U)) \leq \min(\phi_1(W), \phi_2(W)) = \psi(W)$$

Thus, $\psi(U) \leq \psi(W)$.

- **Inductive Step ($\vee$):** Suppose $\psi = \phi_1 \vee \phi_2$. By the inductive hypothesis, assume $\phi_1(U) \leq \phi_1(W)$ and $\phi_2(U) \leq \phi_2(W)$.
The logical OR operator computes the maximum of its inputs.

$$\psi(U) = \max(\phi_1(U), \phi_2(U))$$

Since the inputs are bounded, their maximums are bounded:

$$\max(\phi_1(U), \phi_2(U)) \leq \max(\phi_1(W), \phi_2(W)) = \psi(W)$$

Thus, $\psi(U) \leq \psi(W)$.

This completes the proof of the lemma. ■

Main Proof of Equi-satisfiability

Let F be a formula in NNF. The optimized encoding F_{opt} introduces an auxiliary variable a_ψ for every subformula ψ . Because there are no negations over subformulas in NNF, every proxy variable has strictly positive polarity. Thus, F_{opt} consists of the root unit clause a_F and clauses equivalent to the implication $a_\psi \rightarrow E(\psi)$ for every internal gate.

We must show that F is satisfiable $\iff F_{opt}$ is satisfiable.

1. Forward Direction (F is satisfiable $\implies F_{opt}$ is satisfiable):

Assume F is satisfiable. There exists an assignment V to the original variables that makes F evaluate to 1 (True). We extend V to an assignment V' by setting every auxiliary variable a_ψ to the exact truth value of its corresponding subformula ψ . By definition, $V'(a_\psi) = V'(E(\psi))$, so the

full biconditional $a_\psi \leftrightarrow E(\psi)$ from the standard Tseitin encoding holds. Since F_{opt} only contains the forward implications (a subset of the standard rules), V' satisfies F_{opt} .

2. Reverse Direction (F_{opt} is satisfiable $\implies F$ is satisfiable):

Assume F_{opt} is satisfiable. Let V' be the satisfying assignment for F_{opt} . Let V be the restriction of V' strictly to the original atomic variables.

Because F_{opt} enforces $a_\psi \rightarrow E(\psi)$, we know mathematically that for every subformula, the solver's assignment to the proxy is a *lower bound* on the solver's assignment to the expression:

$$V'(a_\psi) \leq V'(E(\psi))$$

We will prove by **structural induction on ψ** that the proxy assignment is a lower bound for the actual evaluation of the subformula: $V'(a_\psi) \leq V(\psi)$.

- **Base Case (Literals):** If ψ is a literal l , there is no auxiliary variable. The proxy is the literal itself, so $V'(a_l) = V'(l) = V(l)$. Thus, $V'(a_l) \leq V(l)$ holds.
- **Inductive Step ($\wedge$):** Suppose $\psi = \psi_1 \wedge \psi_2$. The encoding enforces $a_\psi \rightarrow (a_{\psi_1} \wedge a_{\psi_2})$, which translates to:

$$V'(a_\psi) \leq V'(a_{\psi_1}) \wedge V'(a_{\psi_2})$$

By the inductive hypothesis, $V'(a_{\psi_1}) \leq V(\psi_1)$ and $V'(a_{\psi_2}) \leq V(\psi_2)$.

Applying the Monotonicity Lemma to the $\wedge$ operator, replacing the proxy inputs with the actual subformula evaluations yields:

$$V'(a_{\psi_1}) \wedge V'(a_{\psi_2}) \leq V(\psi_1) \wedge V(\psi_2) = V(\psi)$$

By transitivity, $V'(a_\psi) \leq V(\psi)$.

- **Inductive Step ($\vee$):** Suppose $\psi = \psi_1 \vee \psi_2$. The encoding enforces $a_\psi \rightarrow (a_{\psi_1} \vee a_{\psi_2})$, translating to:

$$V'(a_\psi) \leq V'(a_{\psi_1}) \vee V'(a_{\psi_2})$$

By the inductive hypothesis, $V'(a_{\psi_1}) \leq V(\psi_1)$ and $V'(a_{\psi_2}) \leq V(\psi_2)$.

Applying the Monotonicity Lemma to the $\vee$ operator, we get:

$$V'(a_{\psi_1}) \vee V'(a_{\psi_2}) \leq V(\psi_1) \vee V(\psi_2) = V(\psi)$$

By transitivity, $V'(a_\psi) \leq V(\psi)$.

Conclusion: By the induction above, the property $V'(a_\psi) \leq V(\psi)$ holds for all subformulas, including the root formula F . Since V' satisfies F_{opt} , the unit clause for the root variable a_F is satisfied, meaning $V'(a_F) = 1$ (True). Substituting this into our proven bound:

$$1 \leq V(F)$$

Therefore, $V(F) = 1$. The assignment V restricted to the original variables makes F True, proving F is satisfiable. ■

Grading Scheme:

- 1 mark for forward direction
- 3 marks for reverse direction - 1.5 marks for core idea + 1.5 marks for structural induction (0.5 marks each for Base Case, OR and AND)

A common mistake was to assume a_i occurring positively meant it will be true in all satisfying assignments. There may be satisfying assignments for the Tseitin-optimized version of the formula in which a_i is assigned false.